Carbon-Taxed Transformers: Efficient, Accurate, and Sustainable LLMs

Ajmain Inqiad Alam
University of Saskatchewan
Saskatoon, Canada
ajmain.alam@usask.ca

Palash R. Roy
University of Saskatchewan
Saskatoon, Canada
palash.roy@usask.ca

Chanchal K. Roy
University of Saskatchewan
Saskatoon, Canada
chanchal.roy@usask.ca

Banani Roy
University of Saskatchewan
Saskatoon, Canada
banani.roy@usask.ca

Kevin A. Schneider
University of Saskatchewan
Saskatoon, Canada
kevin.schneider@usask.ca

Abstract

The growing adoption of Large Language Models in software engineering has introduced unsustainable computational and environmental costs. We propose Carbon-Taxed Transformers (CTT), a compression pipeline inspired by economic carbon taxation, integrating Neural Architecture Search, structured pruning, quantization, and knowledge distillation in a principled sequence. CTT generalizes across encoder-only, encoder-decoder, and decoder-only architectures, and is evaluated on code clone detection, summarization, and generation. Results show up to 49× memory reduction, 10× speedup, and an 81% reduction in CO_2 emissions, while retaining 98% accuracy in clone detection, 89% in summarization, and up to 91% in generation. Ablation studies confirm both the pipeline ordering and each component are essential.

CCS Concepts

• **Computing methodologies** → *Artificial intelligence*; **Machine learning approaches**; • **Software and its engineering** → *Software maintenance tools.*

Keywords

Model Compression, Green Computing, Energy-Efficient Inference

ACM Reference Format:
Ajmain Inqiad Alam, Palash R. Roy, Chanchal K. Roy, Banani Roy, and Kevin A. Schneider. 2026. Carbon-Taxed Transformers: Efficient, Accurate, and Sustainable LLMs. In *34th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (FSE Companion '26), July 05–09, 2026, Montreal, QC, Canada.* 2 pages. https://doi.org/10.1145/3803437.3807387

1 Introduction

Large Language Models (LLMs) have transformed software engineering (SE), enabling advances in different SE tasks [6, 15]. However, their deployment carries significant computational and environmental costs: GPT-3 alone needs 325 GB of GPU memory [10]. Critically, inference accounts for over 90% of total ML energy consumption [7], and a single LLM can emit tons of CO_2 [13].

Model compression techniques such as knowledge distillation (KD) [9], quantization [4], and pruning [5], offer promising directions. However, prior SE works [8, 11, 12] typically apply a single technique to one architecture, leaving combined, multi-architectural compression unexplored. While PEFT methods such as LoRA [2] reduce fine-tuning costs, they retain the full model during inference and thus do not address deployment efficiency.

We propose **Carbon-Taxed Transformers (CTT)**, a compression pipeline that operationalizes a computational *carbon tax*, algorithmic constraints forcing principled trade-offs between performance and efficiency. In this work, we: (1) propose CTT with *architectural generalization* across encoder-only, decoder-only, and encoder-decoder models, (2) introduce *principled pipeline ordering* (NAS→pruning→quantization→KD), validated via ablation studies, (3) enforce *budget-aware optimization* with hard constraints on latency, memory, and CO_2, and (4) demonstrate up to 49× memory reduction, 10× speedup, and 81% CO_2 reduction while retaining 89–98% task performance across three core SE tasks. Further details, including experimental setup and comprehensive analysis, are available in the full paper [1].

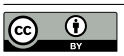
2 Methodology

This section summarizes the methodology from our full paper [1]. CTT targets four key architectural parameters: hidden layers, attention heads, hidden dimensions, and feedforward dimension size (Figure 1 adapted from our full paper [1]).

Stage 1: Structural Reduction. We apply NAS over a predefined configuration space to identify an initial compact architecture, then refine it through structured pruning while monitoring performance. An empirical analysis confirmed that deeper layers often act as near-identity mappings with minimal representational gain, providing principled cut-off points.

Stage 2: Quantization. The pruned model undergoes quantization *before* training, imposing a "tax on precision" which helps to adapt to quantization-induced noise during inference, mitigating the accuracy degradation observed in post-training quantization [4].

Stage 3: KD. The quantized student is trained to match the teacher's soft output distribution via KD loss, acting as a "performance rebate" that recovers accuracy lost during the compression.



FSE Companion '26, Montreal, QC, Canada

ACM ISBN 979-8-4007-2636-1/2026/07
https://doi.org/10.1145/3803437.3807387

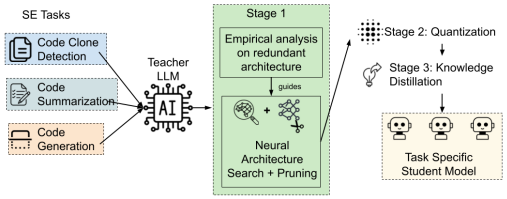


Figure 1: CTT pipeline (adapted from our full paper [1]).

Table 1: Inference-time CO_2 emissions.

Task	Model	Size	CO_2 (kg)	Δ
CCD	UniXCoder (Teacher)	504 MB	0.00421	–
	CTT Student	**1.6 MB**	**0.00198**	↓53%
Summ.	CodeT5-base (Teacher)	892 MB	0.00936	–
	CTT Student	**65 MB**	**0.00549**	↓41%
Gen.	CodeGen-350M (Teacher)	1.4 GB	0.03455	–
	CTT Student	**52.5 MB**	**0.00669**	↓81%
	Qwen 1.5B (Teacher)	3.1 GB	0.04300	–
	CTT Student	**216 MB**	**0.01318**	↓69%

3 Results and Discussion

We evaluate CTT on code clone detection (CCD) using BigCloneBench [14] and UniXCoder as teacher (encoder-only), code summarization using CodeSearchNet summarization dataset [3] and CodeT5-base [15] (encoder-decoder), and code generation using CodeSearchNet (code generation)+MBPP and CodeGen-350M and Qwen Coder 2.5 1.5B (decoder-only). All results and discussion reported in this section are summarized from our full paper [1].

Performance. The CTT student retains 98% of teacher performance in CCD while being 300× smaller. It also outperforms CodeBERT and CodeT5 in recall while matching CodeT5+ and Compressor [12], and outperforms Avatar [11] in precision. For summarization, it preserves 89% ROUGE-L at 92.7% size reduction, outperforming CodeT5-small across BLEU, METEOR, and ROUGE-L. In generation, the CodeGen-derived student maintains 91% BLEU at 97% reduction; the Qwen-derived student retains 80% BLEU at 93% reduction, with pass@1 reaching 68% of the teacher's.

Efficiency. CTT achieves up to 8× GPU and 10× CPU inference speedups in CCD, 3× in summarization, and 3–6× in generation. Memory reductions reach up to 49× on GPU and 8× on CPU.

Environmental Impact. Table 1 summarizes inference-time CO_2 emissions from our full paper [1], measured using CodeCarbon. CTT students consistently emit substantially less CO_2.

Ablation Insights. Two ablation studies validate CTT's design. First, our NAS→pruning→quantization ordering outperforms all permutations as alternative orderings increase latency by 4–6% and CO_2 by up to 25%. Second, removing KD causes accuracy to collapse to ∼52% (near-random), confirming that compression delivers efficiency while KD restores task-specific knowledge. Across all tasks, the average teacher–student gap is +2.1 points (95% bootstrap CI [+0.0, +4.3]; paired t-test $p = 0.23$), showing no statistically significant degradation. CTT's one-time training overhead is offset by inference savings that dominate long-term energy costs [7].

4 Conclusion

CTT is a compression pipeline integrating NAS, pruning, quantization, and KD that yields compact, fast, environmentally aligned SE models: up to 49× memory reduction, 10× speedup, and 81% CO_2 reduction while retaining 89–98% task performance. Its composable ordering and budgeting give developers fast on-device tools, enterprises low-cost deployment, and researchers compact models under modest resources.

5 Acknowledgment

This research is supported in part by the Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery Grants program, the Canada Foundation for Innovation's John R. Evans Leaders Fund (CFI-JELF), and by the industry-stream NSERC CREATE in Software Analytics Research (SOAR).

References

[1] Ajmain Inqiad Alam, Palash R. Roy, Chanchal K. Roy, Banani Roy, and Kevin A. Schneider. 2026. Carbon-Taxed Transformers: A Green Compression Pipeline for Overgrown Language Models. In *Proceedings of the 34th ACM International Conference on the Foundations of Software Engineering (FSE) (Proceedings of the ACM on Software Engineering (PACMSE), Vol. 3)*. ACM, Article FSE047.

[2] Edward J Hu, yelong shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2022. LoRA: Low-Rank Adaptation of Large Language Models. *International Conference on Learning Representations* (2022). https://openreview.net/forum?id=nZeVKeeFYf9

[3] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).

[4] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. 2018. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2704–2713.

[5] Paul Michel, Omer Levy, and Graham Neubig. 2019. Are sixteen heads really better than one? *Advances in neural information processing systems* 32 (2019).

[6] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. In *The Eleventh International Conference on Learning Representations*. https://openreview.net/forum?id=iaYcJKpY2B_

[7] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. Carbon emissions and large neural network training. *arXiv preprint arXiv:2104.10350* (2021).

[8] Mootez Saad, José Antonio Hernández López, Boqi Chen, Dániel Varró, and Tushar Sharma. 2024. ALPINE: An adaptive language-agnostic pruning method for language models for code. *arXiv preprint arXiv:2407.04147* (2024).

[9] Victor Sanh, Lysandre Debut, et al. 2019. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. In *5th Workshop on Energy Efficient Machine Learning and Cognitive Computing, NeurIPS*.

[10] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Daniel Y. Fu, Zhiqiang Xie, Beidi Chen, Clark W. Barrett, Joseph Gonzalez, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. High-throughput Generative Inference of Large Language Models with a Single GPU. In *International Conference on Machine Learning*.

[11] Jieke Shi, Zhou Yang, Hong Jin Kang, Bowen Xu, Junda He, and David Lo. 2024. Greening large language models of code. In *Proceedings of the 46th international conference on software engineering: software engineering in society*. 142–153.

[12] Jieke Shi, Zhou Yang, Bowen Xu, Hong Jin Kang, and David Lo. 2022. Compressing pre-trained models of code into 3 mb. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*. 1–12.

[13] Emma Strubell, Ananya Ganesh, and Andrew McCallum. 2020. Energy and policy considerations for modern deep learning research. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 13693–13696.

[14] Jeffrey Svajlenko and Chanchal K Roy. 2021. Bigclonebench. In *Code Clone Analysis: Research, Tools, and Practices*. Springer, 93–105.

[15] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C.H. Hoi. 2021. CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 8696–8708.